INTRODUCTION

What is Django?

Django is a **high-level Python web framework** used to build **secure, scalable, and dynamic web applications** quickly.

It follows the **MVT (Model–View–Template)** architecture and comes with many **built-in features**, so developers don't have to write everything from scratch.

Key Points

- Written in **Python**
- Follows **MVT architecture**
- Used for building the web applications

Examples of applications built using Django

- E-commerce websites
- Learning Management Systems (LMS)
- REST APIs (using Django REST Framework)
- Banking & enterprise applications

Why Django?

Django is preferred because it is **fast, secure, and powerful**.

Advantages of Django

1. **Fast Development**
    - Comes with built-in features like:
        - Admin panel
        - Authentication

        - ORM
    - Less code, faster output

2. **Secure**
3. **Built-in Admin Panel**
    - Automatically generated admin interface
    - Easy to manage database records
4. **ORM (Object Relational Mapper)**
    - No need to write raw SQL
    - Database-independent
5. **Scalable**
    - Used by big companies
    - Can handle large traffic

Companies using Django

- Instagram
- YouTube
- Spotify
- Mozilla

1. Command

A **command** is an **instruction** given to the computer to perform a **specific task**.

Commands are usually **single statements**.

Examples (Python)

- `print()` → Used to display output on the screen
- `input()` → Used to take input from the user

Each command performs **one action** at a time.

---

2. Program

A **program** is a **set (sequence) of commands** written together to perform a **specific task**.

In simple words:

> **Multiple commands combined together form a program.**

Example: Program to Add Two Numbers (Python)

```
a = int(input())
b = int(input())
c = a + b
print(c)
```

Explanation:

- `input()` → Takes input from the user
- `int()` → Converts input to integer
- `+` → Adds two numbers
- `print()` → Displays the result

✔ Here, **multiple commands** work together to complete one task, so this is called a **program**.

3. Software

Software is a **collection of programs or instructions** that are **interrelated and interdependent**, and they **work together** to achieve a **specific task or goal**.

In simple words:

> **Software = Multiple programs working together**

Relation Between Command, Program, and Software

- **Command → Single instruction**
- **Program → Group of commands**
- **Software → Collection of programs**

Types of Application/Software

1. IOS Application

An iOS application is a type of mobile application designed specifically for Apple devices such as iPhone and iPad.
These applications run on Apple's iOS operating system.

Examples

Using Internet:

- **WhatsApp**
- **Swiggy**
- **Instagram**
- **LinkedIn**

Without Internet:

- **Contacts**
- **Messages**
- **Camera**

---

2. Android Application

An Android application is a mobile application designed to run on devices that use the Android operating system, such as smartphones and tablets.

Examples

Using Internet:

- **WhatsApp**
- **Swiggy**

- **Instagram**
- **LinkedIn**

Without Internet:

- **Contacts**
- **Messages**
- **Camera**

---

3. Desktop Application

A desktop application is software that is installed and executed on a personal computer or laptop.
These applications do not require a web browser to function.

Examples

Using Internet:

- **WhatsApp Desktop**
- **Instagram Desktop**
- **ChatGPT Desktop**
- **MS Teams**

Without Internet:

- **MS Word**
- **Paint**
- **PowerPoint**
- **Camera**

---

4. Web Application

A web application is a type of software that runs inside a web browser.
It does not require installation on the user's system and usually requires an internet connection.

Examples

- **Amazon**
- **Flipkart**
- **Gmail**
- **Google Docs**

Architecture

Architecture refers to the **fundamental structure of a system**.
It explains:

- How different **components** are organized
- How these components **communicate** with each other

In simple words:

Architecture = Design of the system

Web Application Architecture

Web application architecture describes how a web application is designed and structured.

It explains:

- **How many layers are used**
- **Where data is stored (database)**
- **Where business logic is written**
- **How the output is displayed to the user**

Main Components in Web Application Architecture

1. **Client (Frontend)**
    - **Browser or mobile app**
    - **Displays output to the user**
2. **Server (Backend)**
    - **Handles requests**
    - **Contains business logic**
3. **Database**
    - **Stores application data**

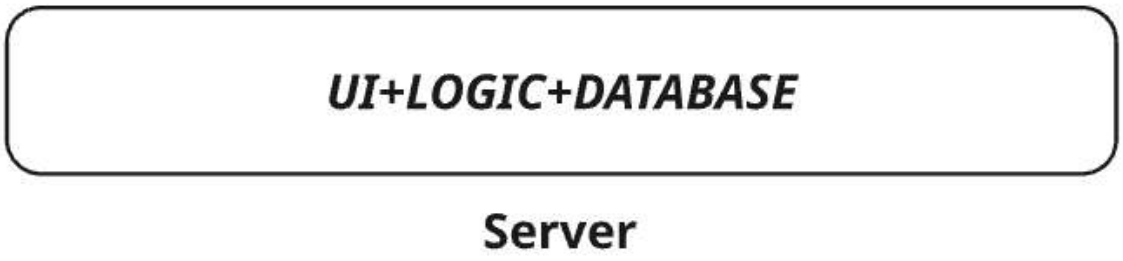
Why Web Application Architecture is Important

- **Improves performance**
- **Makes application scalable**
- **Helps in maintenance**
- **Increases security**

One-Line Explanation

> **Web application architecture explains the structure of a web application and how its components interact with each other.**

1. One-Tier Architecture



In 1-Tier Architecture, all components of the application are present in a single layer.

- **User Interface**
- **Business Logic**
- **Database**

All run on the same system.

Example

- **Calculator**
- **MS Word**

- **Simple offline applications**

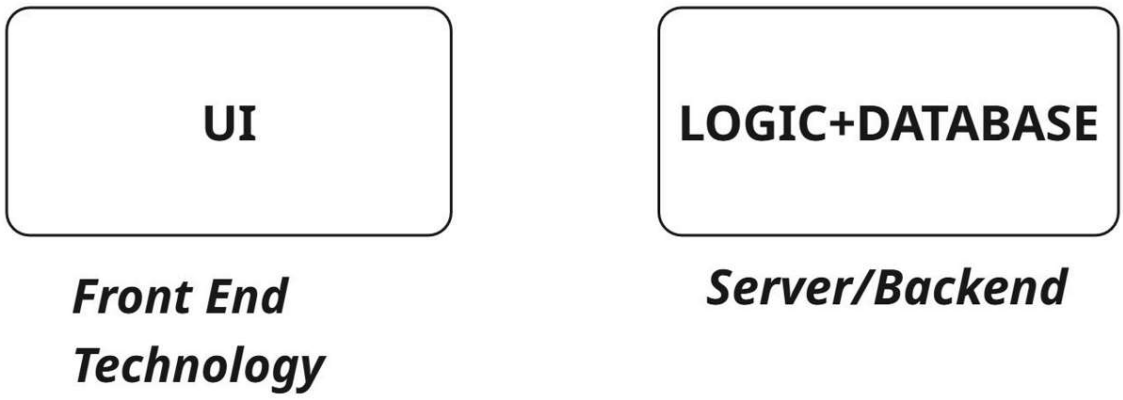
Advantages

- **Simple to develop**
- **Easy to use**

Disadvantages

- **Not scalable**
- **Low security**
- **Not suitable for web applications**

2. Two-Tier Architecture



In 2-Tier Architecture, the application is divided into two layers:

1. **Client Layer**
    - **User interface**
2. **Server Layer**
    - **Business logic**
    - **Database**

Example

- **Desktop applications connected to a database**
- **Small internal applications**

Advantages

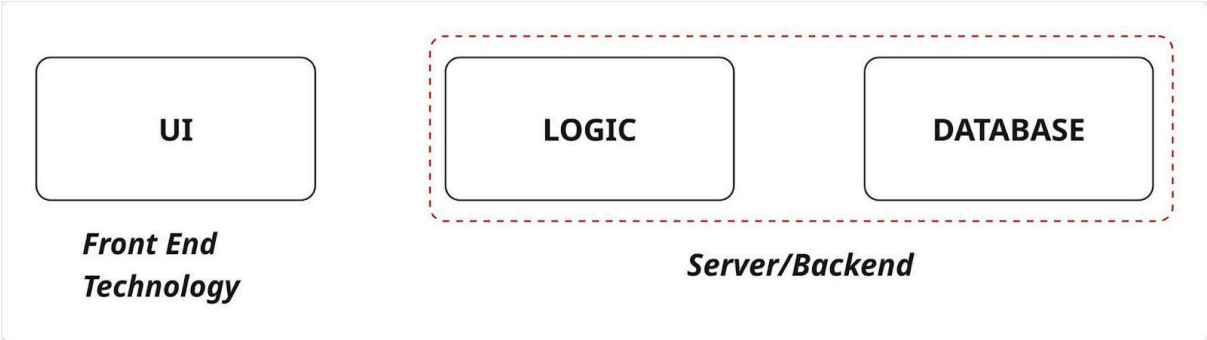
- **Faster than 1-tier**
- **Simple architecture**

Disadvantages

- **Less secure**
- **Limited scalability**
- **Database is directly exposed**

---

3. Three-Tier Architecture



In 3-Tier Architecture, the application is divided into three layers, each with a specific role.

1. **Presentation Layer**
    - **User interface (Browser, Mobile App)**
2. **Application Layer (Business Logic)**

    - **Handles logic and processing**

3. **Data Layer**
    - **Database (MySQL, PostgreSQL)**

Example

- **Web applications**
- **E-commerce websites**
- **Django applications**

Why Does Django Use 3-Tier Architecture?

- **Better security**
- **High scalability**
- **Easy maintenance**
- **Separation of concerns**
